

Evidence

Verbatim output from every step that has been run. Nothing on this page is typed by hand, and all of it can be reproduced from a fresh clone.

Live testnet, tenant `did:t3n:f21dce7928980eeee7dc93618b91f602a80fe1c4`.

Every block below was also captured to a file at the moment it ran. Those files are in `submission/evidence/`, numbered in the order they were produced, and the submission PDF quotes them directly. So each claim here has a source you can open.

Two notes on the final state of the account.

The receipt this page verifies — `rcpt_8fb056c99bbea7e655d72075`, contract 639 — is the one committed at `submission/live-receipt.json`. A later run registered contract 653 and issued `rcpt_03cc7f7a193196184c661f0d`, which also verified in-enclave; that run's output is in the evidence folder.

That later run stopped at step 11 because the 20,000 credit grant ran out. The node now answers `InsufficientCredit (required=1000000000, available=0)`. This is recorded as finding T-15, and it is worth noticing what it does *not* break: the offline verification in section 10 still passes, because recomputing a digest needs no credits, no session, and no network.

1. The contract builds and its tests pass

```
$ cargo test --target x86_64-pc-windows-gnu
test result: ok. 53 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
   Doc-tests z_tenant_nullstamp
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

$ cargo build --target wasm32-wasip2 --release
   Finished `release` profile [optimized] target(s) in 9.83s
252806 bytes   target/wasm32-wasip2/release/z_tenant_nullstamp.wasm
```

No compiler warnings. Well inside the tenant quota of `max_wasm_bytes: 1048576`.

2. The component is valid and its capability surface is minimal

```
$ wasm-tools component wit target/wasm32-wasip2/release/z_tenant_nullstamp.wasm

world root {
  import host:tenant/tenant-context@1.0.0;
  import host:interfaces/logging@2.1.0;
  import host:interfaces/kv-store@2.1.0;
  import host:interfaces/http-with-placeholders@2.1.0;
  ...
  export z:tenant-nullstamp/contracts@0.1.7;
}
```

Two absences are deliberate and worth stating.

Plain `http` is not imported. Nullstamp has exactly one way out — `http-with-placeholders` — so no path inside this contract can send data without passing through host-side marker resolution. That is a design choice, not a default.

`signing@2.1.0` is not imported either, but for a different reason: importing it prevents the contract from being instantiated at all. See finding T-12.

`wit-bindgen` also trims imports to what is actually called. In the built component `logging` narrows to `info`, and `kv-store` narrows to `get`, `put`, `set-claims-digest`, and `scan`. Since the host rejects a contract that asks for anything outside its world, that narrow surface is an enforced boundary rather than tidiness.

3. The testnet is reachable and its attestation verifies

Runs without a developer key.

```
$ npm run preflight

SDK environment
[ok]   SDK default environment – testnet
[ok]   node address – https://cn-api.sg.testnet.t3n.terminal3.io

Node reachability
[ok]   manifest endpoint – HTTP 200

Attestation
[ok]   operator manifest – signature valid, 3 peers, 1 RTMR3 measurement
[ok]   signing time – 2026-08-11T14:14:45Z

SDK crypto component
[ok]   loadWasmComponent – component loaded
```

4. Authentication returns the DID the claim page issued

```
$ npm run step:01

environment: sandbox
anchor verified – 3 peers, 1 RTMR3 measurement
node       : https://cn-api.sg.testnet.t3n.terminal3.io
eth address: 0xcdbf0480addeacbe6e7b27154585db65ad249379
tenant DID : did:t3n:f21dce7928980eeea7dc93618b91f602a80fe1c4
```

The DID matches the one printed on the claim page character for character, which confirms the derivation path from API key to Ethereum address to DID.

Note that this ran with the corrected client configuration from findings T-01 and T-02. It worked on the first attempt.

5. Tenant state and quotas

```
$ npm run step:02

status : active
label  : testnet-dev

tenant already active – the SSO claim page had already done it.
re-claim skipped, because tenant.claim() on an active tenant answers 500.

credit balance:
  available : 3502747402
  reserved  : 0

quotas that bound the work ahead:
  max_contracts      : 10
  max_maps           : 50
  max_wasm_bytes     : 1048576
  outbox_calls_per_minute_max : 10
  fuel_per_call_max  : 50000000
```

The skip is not laziness — see finding T-11.

6. Registration, maps, credential, grant

```
$ npm run step:03
size      : 252806 bytes
registering nullstamp version 0.1.7...
canonical name : z:f21dce7928980eeea7dc93618b91f602a80fe1c4:nullstamp
contract id   : 639

$ npm run step:04
created : z:f21dce...:secrets  -> active
created : z:f21dce...:receipts -> active

$ npm run step:04b
effective contract id : 639
  readers & writers now restricted to contract 639

$ npm run step:05
seeded  : z:f21dce...:secrets / upstream_api_key
read back, length 36 characters

$ npm run step:06
functions      : issue-receipt, verify-receipt, list-receipts
allowed hosts  : postman-echo.com
validFromSecs  : 1786535847
validUntilSecs : 1794311847
```

Step 04b exists because of finding T-13: contract ids are minted per registration, and map ACLs bind to the id.

7. A receipt issued inside the enclave

```
$ npm run step:07

target      : https://postman-echo.com/post
declared    : first_name, last_name
body sent   : {"reason":"nullstamp_demo",
               "first_name":"{{profile.first_name}}",
               "last_name":"{{profile.last_name}}"}

{
  "core": {
    "contract_id": 639,
    "contract_version": "0.1.7",
    "extracted_pointers": [ "/json/reason" ],
    "fields_used": [ "first_name", "last_name" ],
    "issued_at_secs": 1786554217,
    "method": "POST",
    "purpose": "receipt_issuance_demo",
    "request_body_sha256": "26e5ee62768086e9213455818dbc5d17b41356f41f3d36ac92979a9059cffc6b",
    "response_body_sha256": "af8e3bce0cd50618030d03a18241046b5b8c8e41c173b6400eb754568a8528f2",
    "response_code": 200,
    "schema": "nullstamp.receipt.v1",
    "seq_no": 114186,
    "subject_did": "f21dce7928980eeea7dc93618b91f602a80fe1c4",
    "target_host": "postman-echo.com",
    "target_url": "https://postman-echo.com/post",
    "tenant_did": "f21dce7928980eeea7dc93618b91f602a80fe1c4"
  },
  "digest_sha256": "8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3",
  "extracted": { "/json/reason": "nullstamp_demo" },
  "receipt_id": "rcpt_8fb056c99bbea7e655d72075",
  "signature": null,
  "signing_error": "signing capability not imported; integrity rests on the claims digest"
}
```

Several things are load-bearing here.

`response_code` is 200, so the outbound call really left the enclave and reached the upstream. `extracted` shows the response was parsed, which is only possible if the round trip happened. `seq_no` and `issued_at_secs` come from the cluster, not from my machine.

And `core` lists only field **names**. The values are absent and cannot be present, because the code that assembles this section never receives them.

8. Verification inside the enclave

```
$ npm run step:08

checking: rcpt_8fb056c99bbea7e655d72075
anchor verified - 3 peers, 1 RTMR3 measurement(s)
{
  "digest_sha256": "8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3",
  "reason": null,
  "receipt_id": "rcpt_8fb056c99bbea7e655d72075",
  "valid": true
}

digest matches, receipt is valid
```

Note that this is the contract recomputing the digest from the row it stored, not replaying a cached answer. Section 10 does the same arithmetic outside the node and lands on the same value.

9. The trail, and the contract's own log

```
$ npm run step:09

count: 7
rcpt_477609b57a71786d34613857 | POST postman-echo.com | 200 | ["first_name","last_name"]
rcpt_4d8790a69bad4bbfd53c757f | POST postman-echo.com | 200 | ["first_name","last_name"]
rcpt_51e9569d673ac2fc7e154fa2 | POST postman-echo.com | 200 | ["first_name","last_name"]
rcpt_5bd9c8970986c53d5ec82043 | POST postman-echo.com | 200 | ["first_name","last_name"]
rcpt_6dc44359e21d265d61bde90 | POST postman-echo.com | 200 | ["first_name","last_name"]
rcpt_8fb056c99bbea7e655d72075 | POST postman-echo.com | 200 | ["first_name","last_name"]
rcpt_b95858dd44433780f8b45abb | POST postman-echo.com | 200 | ["first_name","last_name"]

$ npm run step:10

{
  "entries": [
    {
      "level": "info",
      "message": "nullstamp: calling POST postman-echo.com for receipt_issuance_demo with 2 profile fields"
    },
    {
      "level": "info",
      "message": "nullstamp: receipt rcpt_8fb056c99bbea7e655d72075 issued, upstream status 200"
    }
  ],
  "next_seq": 1,
  "truncated": false
}
```

The seven receipts were issued across successive registrations of the same contract, and the newest deployment reads every one of them.

The log names the field count, never the field values. That is the discipline the whole design exists to enforce, and it holds even in the debugging surface.

Getting `list-receipts` to this state required finding T-14: `scan` returns raw CAS pointers where `get` resolves them.

10. The central claim: a live receipt verified outside the node

This is the part that matters, so it is not asserted — it is run.

```
$ npm run step:11
saved to      : submission/live-receipt.json
receipt_id   : rcpt_8fb056c99bba7e655d72075
digest       : 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
target host  : postman-echo.com
fields used  : ["first_name", "last_name"]
seq_no       : 114186

$ npm run verify:offline -- ../submission/live-receipt.json

receipt_id    : rcpt_8fb056c99bba7e655d72075
digest recorded : 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
digest computed : 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
signature      : none – signing capability not imported; integrity rests on the claims digest

RESULT: valid. Digest recomputed outside the node and it matches.
exit=0
```

The verifier imports no SDK, opens no session, and makes no network request. It reads a JSON file and computes SHA-256 over the canonical form.

Both sides agree because both build the canonical form by the same rule: object keys ascending, no whitespace, array order preserved. On the Rust side that comes from `serde_json` resting on `BTreeMap`; on the TypeScript side it is ten lines.

The same check runs in any browser at nullstamp.vercel.app/verify, using Web Crypto, with no request to us.

11. Tampering is rejected

Three attempts, all against the live receipt above.

Hiding a field that was actually used. The most realistic cheat: claiming less data than you touched. Here `fields_used` is cut from two entries to one.

```
$ npm run verify:offline -- ../submission/tampered-fields.json

RESULT: invalid.
- digest mismatch: recorded 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3,
                    recomputed 9f2c28bea15b65b3c3507864989044b969ac7f750e0f36c8ed8a92e7e8f561ef
- receipt_id does not derive from digest: recorded rcpt_8fb056c99bba7e655d72075,
                    expected rcpt_9f2c28bea15b65b3c3507864

exit=1
```

Swapping the destination host to `api.attacker.com`.

```
$ npm run verify:offline -- ../submission/tampered-host.json

RESULT: invalid.
- digest mismatch: recorded 8fb056c99bba7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3,
                    recomputed 37f7ce215e46aad9c83d4ae90aa32a44e3fe237dd96d85fd1ead935de7e8b8c1
- receipt_id does not derive from digest

exit=1
```

Exit codes were measured directly, not through a pipe: the valid receipt exits 0, both tampered files exit 1.

Editing the content and repairing the digest to match. The careful version of the attack. It still fails, because the receipt id derives from the digest and cannot be repaired without becoming a different id:

```
$ cargo test digest_yang_dipalsukan_agar_cocok_tetap_gagal_di_identitas
test verify::tests::digest_yang_dipalsukan_agar_cocok_tetap_gagal_di_identitas ... ok
```

12. What is not proven here

The receipt above was issued by a tenant contract, where the calling user profile is `None`. So while the request body carried `{{profile....}}` markers and the upstream returned 200, this run does not demonstrate markers being **substituted** with real values — it demonstrates that the contract never holds them.

Proving substitution end to end needs a user session with a populated profile, which is the natural next step for this contract and is not covered by the onboarding brief.

Everything else on this page is a live result.